

TC2006B: Device Communications

Network Services: DHCP, DNS, HTTP, and FTP

Juan David Guarnizo Hernandez, PhD

Computer Science Department
Tecnológico de Monterrey
FJ2026

Agenda

- 1 Overview
- 2 Dynamic Host Configuration Protocol (DHCP)
- 3 Domain Name System (DNS)
- 4 HyperText Transfer Protocol (HTTP)
- 5 File Transfer Protocol (FTP)

Common Application Layer Protocols

Protocol	Transport	Port(s)	Primary Function
HTTP / HTTPS	TCP	80 / 443	Web document transfer (unencrypted / secure).
DNS	UDP / TCP	53	Translates human-readable domain names to IP addresses.
SMTP	TCP	25, 587	Email routing and delivery between mail servers.
IMAP / POP3	TCP	143 / 110	Email retrieval from a mail server by a client.
SSH	TCP	22	Secure, encrypted remote command-line access.
FTP	TCP	20, 21	File transfer (Port 21 for Control, Port 20 for Data).
DHCP	UDP	67, 68	Dynamic and automatic assignment of IP addresses.

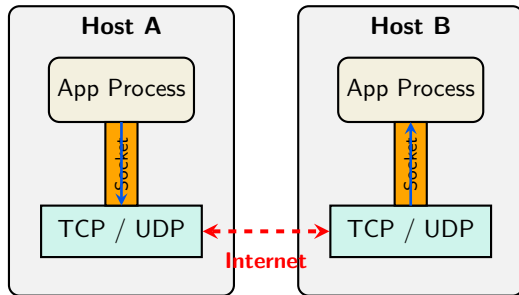
Processes Communicating

The Doorway to the Network

- A **process** is a program running within a host.
- Processes communicate by exchanging messages through a software interface called a **socket**.

Addressing Processes

To receive messages, a process must have an identifier: **IP Address + Port Number**.

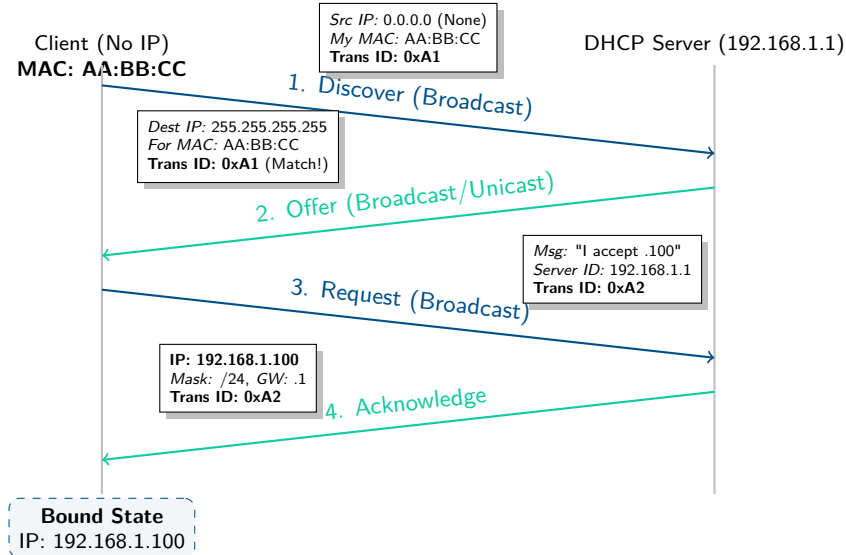


Manually configuring thousands of hosts is impossible. DHCP automates IP, Mask, Gateway, and DNS assignment.

Discovery Process (DORA):

- ➊ **Discover:** "Is there a DHCP server?" (Broadcast)
- ➋ **Offer:** "I have IP 192.168.1.100 for you." (Broadcast/Unicast)
- ➌ **Request:** "I'll take it!" (Broadcast to notify all servers)
- ➍ **Ack:** "Confirmed. Valid for 8 hours."

DHCP Exchange Operation



Domain Name System (DNS)

It works as a directory service for the internet.

- Humans prefer identifiers like `www.google.com`.
- Routers prefer fixed-length, hierarchical numbers like `142.250.190.36`.
- **DNS** translates between the two.

DNS Features

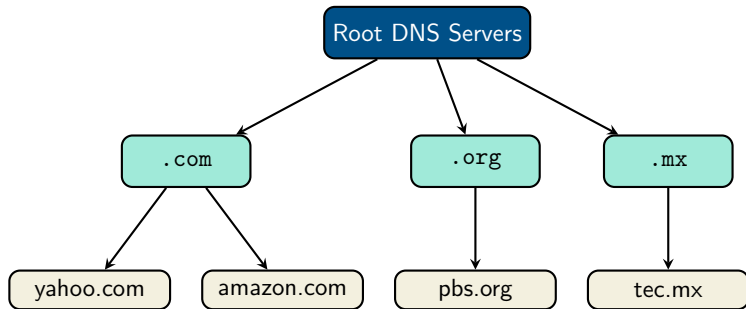
- A **distributed** database implemented in a hierarchy of many name servers
- An application-layer protocol allowing hosts to query the database
- Runs over UDP on Port 53

Why not centralize DNS?

If there was only one DNS server in the world:

- Single point of failure
- Massive traffic volume
- Distant centralized database (high delay)

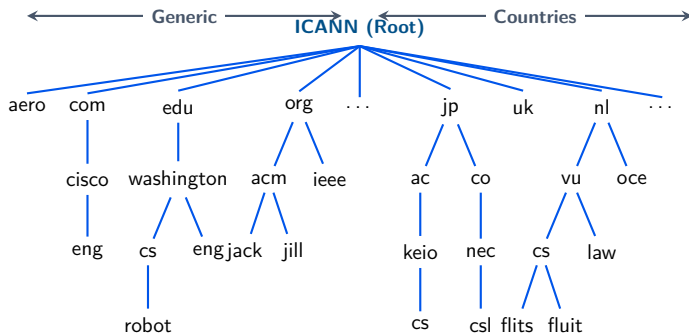
A Distributed, Hierarchical Database



- **Root Name Servers:** Contacted by local name servers; returns IP of TLD servers.
- **Top-Level Domain (TLD) Servers:** Responsible for generic (com, org, edu) and country (us, nl, jp) domains.
- **Authoritative Name Servers:** Organization's own DNS server, providing authoritative hostname to IP mappings.

The Domain Name Space

- **Management:** The root of the hierarchy is managed by **ICANN** (Internet Corp. for Assigned Names and Numbers), established in 1998.
- **Structure:** The Internet is divided into over **250 Top-Level Domains (TLDs)**, which are partitioned into subdomains to form a massive tree.
- **Leaves:** A leaf domain (one with no subdomains) can represent a single host or an entire company containing thousands of hosts.



DNS Resource Records (RR)

The DNS database maps domain names to resource records (a 5-tuple).

Resource Record Format

Domain_name Time_to_live Class Type Value

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

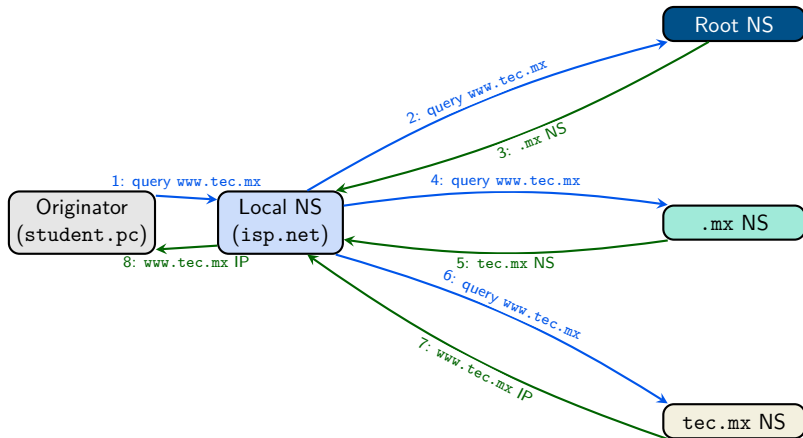
DNS Database Example

Authoritative Database data for cs.vu.nl

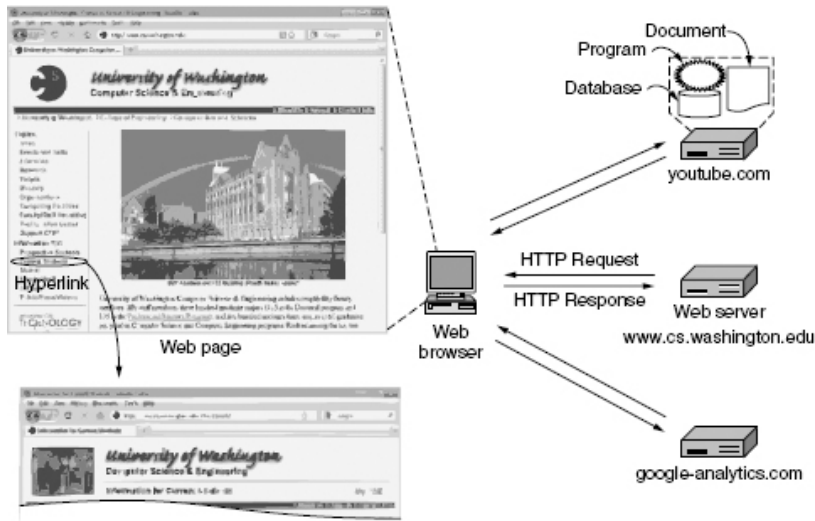
cs.vu.nl.	86400	IN	SOA	star boss (9527,7200,7200,241920,86400)
cs.vu.nl.	86400	IN	MX	1 zephyr
cs.vu.nl.	86400	IN	MX	2 top
cs.vu.nl.	86400	IN	NS	star
star	86400	IN	A	130.37.56.205
zephyr	86400	IN	A	130.37.20.10
top	86400	IN	A	130.37.20.11
www	86400	IN	CNAME	star.cs.vu.nl
ftp	86400	IN	CNAME	zephyr.cs.vu.nl
flits	86400	IN	A	130.37.16.112
flits	86400	IN	A	192.31.231.165

DNS Iterative Query

Iterative Query: The contacted server replies with the name of the *next* server to contact ("I don't know this name, but ask this server.")



The Web and HTTP



The World Wide Web has evolved into the dominant service running over the Internet

HyperText Transfer Protocol (HTTP)

It is the protocol that rules the web supporting the exchange of information between clients and servers.

Key Characteristics

- **Client/Server Model:**
 - *Client:* Browser that requests, receives, and displays Web objects.
 - *Server:* Web server sends objects in response to requests.
- **Transport Protocol:** Uses TCP (usually port 80).
- **Stateless:** The server maintains no information about past client requests.

Why Stateless?

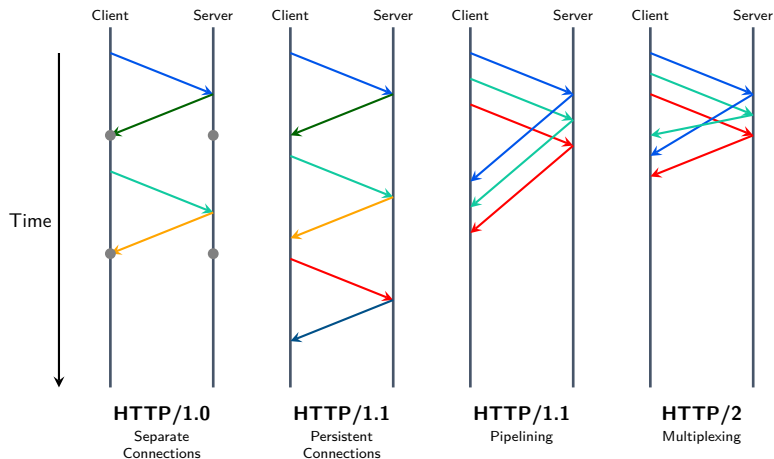
Maintaining state requires complex history management. If a server crashes, its state must be reconciled with the client. Stateless protocols are much easier to scale!

Evolution of HTTP

HTTP was created at CERN in 1989 and is now maintained by the Internet Engineering Task Force (IETF).

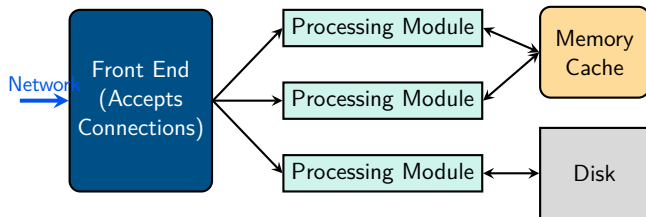
Version	Title	Key Additions
0.9 (1991)	The One-Liner	GET method only.
1.0 (1996)	The Expansion	Request headers, version field, status codes, content type, POST, HEAD.
1.1 (1997)	The Standard-Bearer	Host header, persistent connections, pipelining, PUT, PATCH, DELETE, CONNECT, TRACE, OPTIONS.
2.0 (2015)	The Need for Speed	Request multiplexing over single connection, request prioritization, automatic compressing, server push.
3.0 (2022)	The Next Generation	Changes transport layer from TCP to QUIC .

Evolution of HTTP Connections



Web Server Architecture

A simple server processes one request at a time, often bottlenecking on slow disk I/O. Modern servers are **multithreaded** to handle thousands of concurrent requests.



Maintaining State: Cookies & Session Tokens

HTTP is inherently **stateless**; yet web and mobile apps are inherently stateful. The primary workaround is to pass a "state identifier" with every single request (e.g., **SessionID** or **Cookies**).

Cookie Fields	
Domain	The server domain the cookie originated from (e.g., <code>amazon.com</code>).
Path	The directory path on the server where the cookie is valid.
Content	A <code>name=value</code> pair containing the actual state data (e.g., a <code>SessionID</code>).
Expires	The timestamp when the cookie should be deleted (Persistent vs. Non-persistent).
Secure	If set, the browser will only send the cookie over an encrypted (HTTPS) connection.

Privacy Concern: *"Third-party" cookies from ad networks can track users across different websites, building extensive behavioral profiles.*

HTML (Structure)

HyperText Markup Language focuses on the *content* and *structure* of the document.

- Uses tags (e.g., `<h1>`, `<b>`, `<img>`, `<form>`).
- Forms allow clients to send input back to the server.

CSS (Presentation)

Cascading Style Sheets focus on the *appearance* of the document.

- Defines fonts, colors, and layout separate from the HTML.
- Enables a consistent look across an entire website via a single `.css` file.

Modern Web pages are often full applications. Content is generated dynamically:

Server-Side Scripting (PHP, JSP, CGI)

- Executes on the server.
- Ideal for interacting with databases (e.g., searching a catalog, processing an order).
- Server generates a completely new HTML page and sends it.

Client-Side Scripting (JavaScript, AJAX)

- Executes in the user's browser.
- Uses the **DOM** (Document Object Model) to change parts of the page instantly.
- **AJAX**: Asynchronously fetches XML/JSON data from the server without reloading the page.

HTTP RTT Analysis

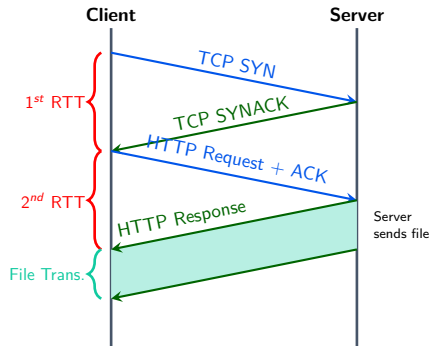
RTT (Round-Trip Time): Time for a small packet to travel from client to server and back.

Non-Persistent HTTP

- At most one object sent over a TCP connection.
- Requires **2 RTTs** per object.
- OS overhead for each TCP connection.

Persistent HTTP (HTTP/1.1+)

- Server leaves connection open after sending response.
- Subsequent HTTP messages pass over the same connection.
- As little as **1 RTT** for all referenced objects.



HTTP Request Format

HTTP messages are written in plain ASCII text.

Example Request Message

```
GET /somedir/page.html HTTP/1.1
```

```
Host: www.someschool.edu
```

```
User-agent: Mozilla/5.0
```

```
Connection: close
```

```
Accept-language: fr
```

```
[Entity Body (empty for GET)]
```

- **Request Line:** Method (GET, POST, HEAD, PUT, DELETE), URL, and HTTP Version.
- **Header Lines:** Provide extra information to the server (e.g., Host, browser type).
- **CRLF (`\r\n`):** Used to indicate the end of lines and the end of the header section.

HTTP Methods

HTTP supports multiple operations, or **methods**, allowing it to function for general-purpose communication (e.g., Web Services, APIs).

Method	Description
GET	Request to read a Web page or resource.
POST	Upload data to the server (e.g., submitting a form).
HEAD	Request the message headers only, without the actual page content.
PUT	Write a new page/resource to the server.
DELETE	Remove the page/resource from the server.
TRACE	Echo the incoming request (used for debugging).

File Transfer Protocol (FTP)

Developed in the 1970s, FTP (RFC 959) is a standard protocol for transferring files between a client and a server on a computer network.

Key Characteristics

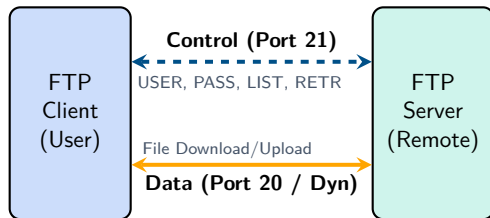
- **Client/Server Model:** The user's interface is the client, interacting with the remote server's file system.
- **Stateful:** Unlike HTTP, the FTP server must maintain state (e.g., current directory, authentication status) for the duration of the session.
- **Cleartext:** Traditionally, FTP sends credentials and data without encryption (though modern variants like FTPS/SFTP secure this).

Out-of-Band Control

The most unique feature of FTP is its **two-connection architecture**. It separates the control commands from the actual data payload.

Two Distinct Ports

- **Port 21 (Control):** A persistent connection used exclusively for sending commands (login, directory changes) and receiving replies.
- **Port 20 (Data):** A temporary connection opened on-demand specifically to transfer file contents or directory listings.



*HTTP sends commands and data over the same connection (**in-band**), whereas FTP sends them separately (**out-of-band**).*

Common Commands (Port 21)

Sent as 4-character ASCII strings:

- **USER / PASS:** Authentication.
- **LIST:** Request a directory listing (sent over the Data connection).
- **RETR:** Retrieve (download) a file.
- **STOR:** Store (upload) a file.
- **PASV:** Tells server to enter Passive mode (client initiates data connection).

Server Replies

Similar to HTTP, FTP uses 3-digit status codes along with an optional phrase:

- **331:** Username OK, password required.
- **125:** Data connection already open; transfer starting.
- **425:** Can't open data connection (often a firewall issue).
- **226:** Closing data connection. File action successful.

FTP Active and Passive Modes

Because FTP uses two separate channels, establishing the Data connection can be tricky, especially when firewalls and NAT are involved.

Active Mode (PORT)

- Client opens a random port for data and tells the server using the PORT command.
- **Server initiates** the data connection to the client.
- *Problem:* Client-side firewalls usually block incoming connections.

Passive Mode (PASV / EPSV)

- Client asks the server to open a port using PASV (or EPSV for IPv6).
- Server opens an ephemeral port and tells the client.
- **Client initiates** the data connection to the server.
- *Solution:* Firewalls allow outgoing connections.

Passive Mode Port Stealing

FTP's separation of control and data, combined with cleartext transmission, creates a severe **Race Condition** vulnerability in Passive mode.

The Hijacking Process

- 1 **Listen:** An attacker on the network sniffs the unencrypted Control channel.
- 2 **Intercept:** The attacker reads the 227 Entering Passive Mode reply containing the server's newly opened ephemeral port.
- 3 **Race:** The attacker rapidly initiates a TCP handshake with the server on that exact port.
- 4 **Steal:** If the attacker connects before the legitimate client, the server binds the data channel to the attacker.

Modern servers use **IP Matching** to mitigate this, dropping data connections if the source IP doesn't match the control

